

Programming Exercises

The programming exercise is divided into five tasks, each focusing on a different Object-Oriented Programming concept. All tasks are implemented using Python to demonstrate the practical application of the concepts covered in Unit 1.

Task 1: Basic Class Hierarchy (Inheritance)

The purpose of inheritance is to reduce code duplication and support the creation of a logical class hierarchy. In the Python code below, inheritance is applied by allowing the Car class to reuse and extend the attributes and methods of the Vehicle class.

```
class Vehicle:
    """Represent a general vehicle."""

    def __init__(self, brand: str, fuel_type: str) -> None:
        self.brand = brand
        self.fuel_type = fuel_type

    def display_details(self) -> str:
        return f"Brand: {self.brand}, Fuel type: {self.fuel_type}"

class Car(Vehicle):
    """Represent a car that inherits from Vehicle."""

    def __init__(
        self,
        brand: str,
        fuel_type: str,
        num_doors: int
    ) -> None:
        super().__init__(brand, fuel_type)
        self.num_doors = num_doors

    def display_details(self) -> str:
        return (
            f"{super().display_details()}, "
            f"Number of doors: {self.num_doors}"
        )

if __name__ == "__main__":
    car = Car("Toyota", "Petrol", 4)
    print(car.display_details())
```

Execution Result

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\code> python.exe .\task1_inheritance.py
Brand: Toyota, Fuel type: Petrol, Number of doors: 4
PS C:\code> 
```

The output confirms that the Car class has successfully inherited the brand and fuel_type attributes from Vehicle while adding the num_doors attribute.

Task 2: Polymorphism with Methods

Polymorphism allows the same method to produce different results depending on the object type. Polymorphism is demonstrated using an abstract Shape class. Both Circle and Rectangle use the area() method, but each class calculates the area differently.

```
from abc import ABC, abstractmethod
from math import pi

class Shape(ABC):
    """Abstract base class for geometric shapes."""

    @abstractmethod
    def area(self) -> float:
        """Calculate and return the shape area."""
        raise NotImplementedError

class Circle(Shape):
    def __init__(self, radius: float) -> None:
        if radius <= 0:
            raise ValueError("Radius must be greater than zero.")

        self.radius = radius

    def area(self) -> float:
        return pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length: float, width: float) -> None:
        if length <= 0 or width <= 0:
            raise ValueError("Length and width must be greater than zero.")

        self.length = length
        self.width = width

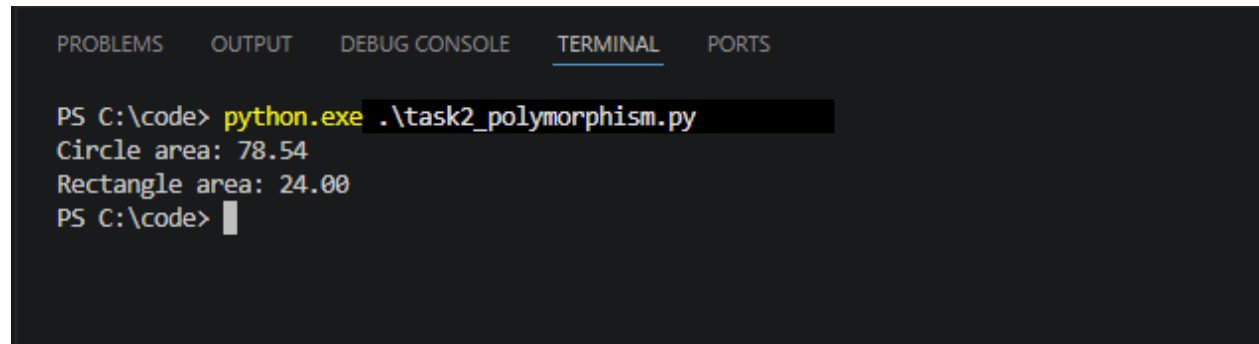
    def area(self) -> float:
        return self.length * self.width
```

```

if __name__ == "__main__":
    shapes: list[Shape] = [
        Circle(5),
        Rectangle(4, 6),
    ]
    for shape in shapes:
        print(f"{shape.__class__.__name__} area: {shape.area():.2f}")

```

Execution Result



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\code> python.exe .\task2_polymorphism.py
Circle area: 78.54
Rectangle area: 24.00
PS C:\code>

```

The output confirms that both Circle and Rectangle use the same area() method but produce different results based on their own implementation.

Task 3: Encapsulation with Access Control

This task demonstrates encapsulation by keeping the account balance private using the `__balance` attribute within the BankAccount class. The balance is modified through the `deposit()` and `withdraw()` methods, which provide controlled access to the object's data.

```

class BankAccount:
    """Represent a bank account with an encapsulated balance."""

    def __init__(self, initial_balance: float = 0.0) -> None:
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative.")

        self.__balance = initial_balance

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("Deposit amount must be greater than zero.")

        self.__balance += amount

```

```

def withdraw(self, amount: float) -> bool:
    if amount <= 0:
        raise ValueError("Withdrawal amount must be greater than zero.")

    if amount > self.__balance:
        print("Insufficient funds.")
        return False

    self.__balance -= amount
    return True

def get_balance(self) -> float:
    return self.__balance

if __name__ == "__main__":
    account = BankAccount(1000)

    account.deposit(250)
    account.withdraw(400)

    print(f"Current balance: {account.get_balance():.2f}")

```

Execution Result

```

PS C:\code> python.exe .\task3_encapsulation.py
Current balance: 850.00
PS C:\code>

```

The output confirms that the account balance was updated through the deposit() and withdraw() methods, resulting in a final balance of 850.00 and demonstrating controlled access to the private balance.

Task 4: Abstraction with Base Class

This task demonstrates abstraction using an abstract Animal class. In the Python code below, the Animal class defines the make_sound() method, while Dog and Cat provide their own versions of it.

```

from abc import ABC, abstractmethod

```

```

class Animal(ABC):
    """Abstract base class for animals."""

    @abstractmethod
    def make_sound(self) -> str:
        """Return the sound made by the animal."""
        raise NotImplementedError

class Dog(Animal):
    def make_sound(self) -> str:
        return "Woof!"

class Cat(Animal):
    def make_sound(self) -> str:
        return "Meow!"

if __name__ == "__main__":
    animals: list[Animal] = [Dog(), Cat()]

    for animal in animals:
        print(f"{animal.__class__.__name__}: {animal.make_sound()}")

```

Execution Result

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\code> python.exe .\task4_abstraction.py
Dog: Woof!
Cat: Meow!
PS C:\code>

```

The output shows that Dog and Cat implement the same `make_sound()` method differently while following the common structure defined by the abstract Animal class.

Task 5: Constructor and Destructor

This task demonstrates the use of constructors and destructors. The `__init__()` method sets up the object when it is created, while `__del__()` defines the behaviour associated with object finalisation.

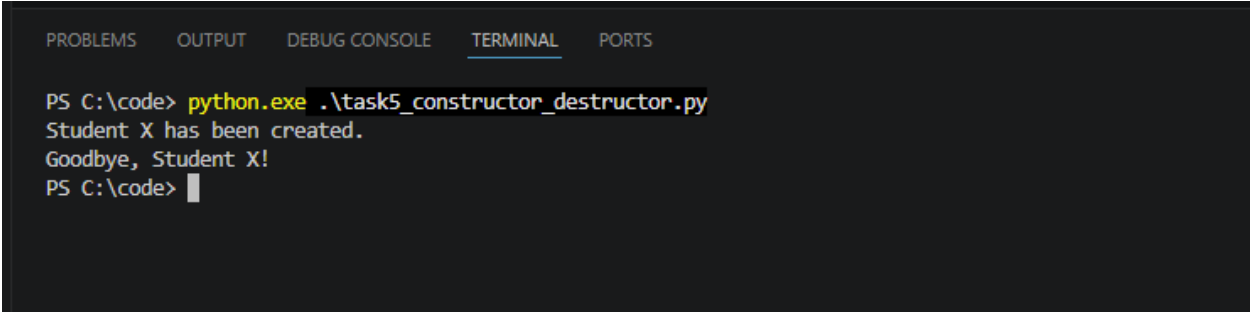
```
class Person:
    """Demonstrate a constructor and destructor."""

    def __init__(self, name: str) -> None:
        self.name = name
        print(f"{self.name} has been created.")

    def __del__(self) -> None:
        print(f"Goodbye, {self.name}!")

if __name__ == "__main__":
    person = Person("Student X")
    del person
```

Execution Result



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\code> python.exe .\task5_constructor_destructor.py
Student X has been created.
Goodbye, Student X!
PS C:\code> █
```

The output shows that `__init__()` was called when the `Person` object was created, while `__del__()` was called after the object was explicitly deleted, demonstrating object initialisation and finalisation.